KNOWLEDGE CORPUS • ARCHITECTURE SPECIFICATION

Cloud Networking & Service Mesh Architectures

A Structured Technical Reference on Container Network Interface (CNI), Overlay Networks, Ingress Controllers, Envoy Proxy, and Istio Service Mesh

Doc ID: GL-CORPUS-03 **Domain:** Cloud Networking **Topic:** CNI & Service Mesh **Target:** Graph RAG System

1. Prerequisites & Fundamental Knowledge

To fully understand cloud networking and service mesh architecture, an engineer must be familiar with standard containerization primitives, Kubernetes workload orchestration, and enterprise computer networking concepts.

The reader should possess a foundational understanding of the following basic concepts before reading this technical document:

- **Linux Network Namespaces & veth Pairs:** Familiarity with virtual Ethernet (veth) interfaces, Linux bridge networking, and network stack isolation, as detailed in GL-CORPUS-01.
- **Kubernetes Workload & Service Model:** Understanding of Pods, Services (ClusterIP, NodePort, LoadBalancer), kube-proxy, and IP-per-Pod networking contracts, as detailed in GL-CORPUS-02.
- **The OSI Model & TCP/IP Stack:** Deep understanding of Layer 2 (Data Link - Ethernet, MAC), Layer 3 (Network - IP, CIDR, Routing), Layer 4 (Transport - TCP, UDP), and Layer 7 (Application - HTTP/1.1, HTTP/2, gRPC, TLS).
- **Public Cloud VPC Networking:** Familiarity with Amazon Web Services Virtual Private Cloud (AWS VPC), subnets, route tables, Elastic Network Interfaces (ENIs), and Internet Gateways.
- **Cryptography & TLS:** Understanding public key infrastructure (PKI), digital certificates, certificate authorities (CAs), and Transport Layer Security (TLS) handshakes.

2. Fundamental Cloud Networking Concepts & Terminology

Before exploring packet lifecycles and sidecar proxy mechanics, we must establish clear definitions for the core networking and service mesh primitives.

Concept: Container Network Interface (CNI)

Beginner-Friendly Definition: The Container Network Interface (CNI) is a standardized plugin specification and library maintained by the Cloud Native Computing Foundation (CNCF) that defines how third-party network plugins must configure network interfaces and allocate IP addresses for Linux containers.

Why It Exists: Kubernetes does not provide built-in network routing implementations. CNI standardizes how orchestrators instruct diverse networking providers (such as Calico, Cilium, Flannel, and AWS VPC CNI) to attach network interfaces when Pods are created.

How It Works: When the Kubelet schedules a Pod, it executes the local CNI binary with JSON configurations (e.g., `ADD` or `DEL` commands). The CNI plugin creates the virtual Ethernet pair, places one interface into the Pod's Network Namespace, assigns an IP address via an IPAM plugin, and configures host routing rules.

Real-World Cloud Example: The AWS VPC CNI plugin running in Amazon EKS clusters to assign native VPC IP addresses directly to Kubernetes Pods from EC2 ENIs.

Related Concepts: Kubelet, Pod Network Namespace, IPAM Plugin, AWS VPC CNI, Calico, Cilium.

Concept: Overlay Network

Beginner-Friendly Definition: An overlay network is a virtual computer network built on top of an underlying physical network (the underlay network). It encapsulates network packets from container workloads inside standard physical transport packets.

Why It Exists: To allow containers distributed across different physical machines and subnets to communicate seamlessly on a unified, flat private IP address space without requiring modifications to the underlying physical network routers and switches.

How It Works: An overlay protocol like VXLAN (Virtual Extensible LAN) or Geneve wraps Layer 2 Ethernet frames inside Layer 4 UDP packets (port 4789). When Pod A sends a packet to Pod B on another node, the source node encapsulates the frame, sends it over the host IP network, and the destination node decapsulates the packet before delivering it to Pod B.

Real-World Cloud Example: Flannel or Calico running VXLAN overlay encapsulation across multi-node Kubernetes clusters on bare-metal or cloud VMs.

Related Concepts: Underlay Network, VXLAN, Geneve, IP-in-IP, MTU Overhead, CNI Plugin.

Concept: Ingress Controller

Beginner-Friendly Definition: An Ingress Controller is a specialized application proxy and Kubernetes controller that manages external HTTP/HTTPS traffic entering the cluster, routing requests to internal backend Services based on domain names (hosts) and URL paths.

Why It Exists: Creating an external Cloud Load Balancer for every single microservice is cost-prohibitive and unmanageable. An Ingress Controller provides a single entry point that consolidates routing rules, TLS termination, and SSL certificates.

How It Works: The controller watches Kubernetes `Ingress` resources via the kube-apiserver, dynamically generates reverse proxy configuration files (e.g., NGINX, HAProxy, or Envoy), and reloads the proxy without dropping active connections.

Real-World Cloud Example: The AWS Load Balancer Controller automatically provisioning an AWS Application Load Balancer (ALB) to route traffic to Amazon EKS Pods.

Related Concepts: Ingress Resource, Reverse Proxy, TLS Termination, AWS ALB, NGINX Ingress Controller.

Concept: Service Mesh & Envoy Proxy

Beginner-Friendly Definition: A Service Mesh is a dedicated infrastructure layer that manages service-to-service (east-west) communication across microservices. Envoy is a high-performance C++ proxy deployed as a sidecar container alongside every application to intercept all inbound and outbound network traffic.

Why It Exists: Application code should not have to manually implement complex networking logic such as mutual TLS encryption, circuit breaking, exponential backoff retries, distributed tracing, and canary traffic splitting.

How It Works: A service mesh divides architecture into a *Data Plane* (Envoy sidecars handling raw traffic) and a *Control Plane* (e.g., Istiod managing certificates, routing policies, and configuration push).

Real-World Cloud Example: Istio Service Mesh managing secure mTLS communication and traffic shifting between payment and inventory microservices in Amazon EKS.

Related Concepts: Istio, Data Plane, Control Plane, Mutual TLS (mTLS), Circuit Breaker, Sidecar Injection.

3. Container Network Interface (CNI) Architecture & Packet Flows

Kubernetes networking adheres to the IP-per-Pod contract, but delegates actual implementation to CNI plugins. CNI plugins fall into two primary architectural categories: **Overlay Networks** and **Underlay** / **Routed Networks**.

3.1 Overlay CNI Plugins (VXLAN & Geneve)

Overlay plugins (such as Flannel and Calico in VXLAN mode) assign Pods IP addresses from a private, non-routable CIDR block that is completely independent of the underlying cloud VPC network.

When Pod A on Node 1 (`10.244.1.5`) sends a packet to Pod B on Node 2 (`10.244.2.8`):

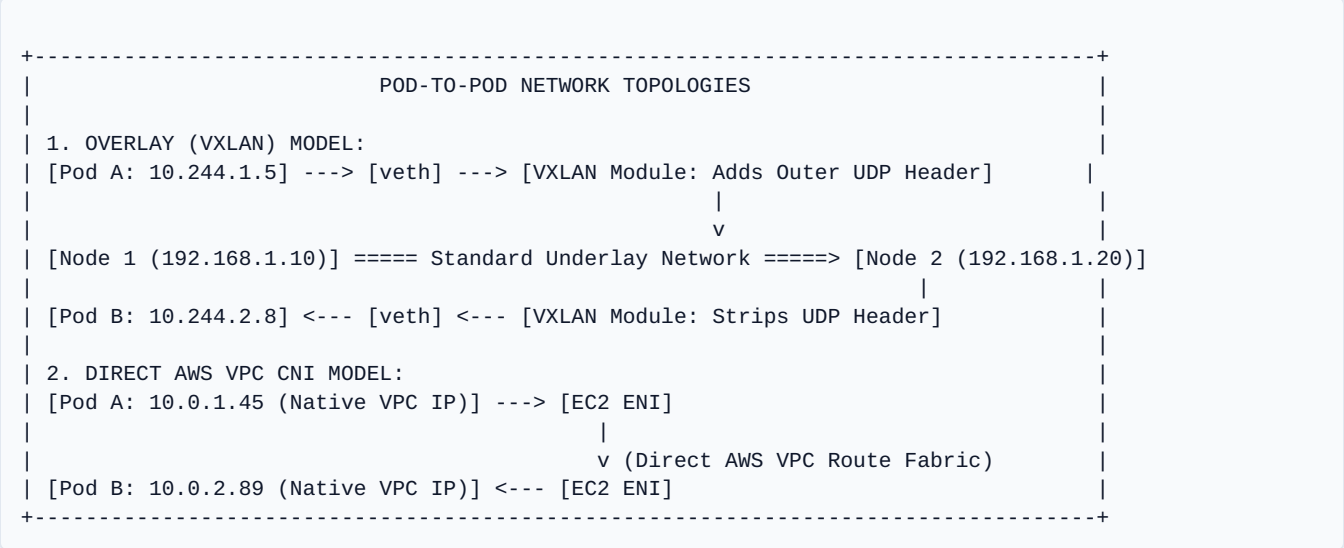
1. The packet exits Pod A's Network Namespace through its `veth` pair onto Node 1.
2. Node 1's kernel detects that the destination IP belongs to a remote Pod CIDR and routes the packet to a virtual VXLAN interface (`vxlan.calico`).
3. The VXLAN module wraps the original IP packet inside an outer UDP packet with Node 1's physical IP as the source and Node 2's physical IP as the destination (UDP port 4789).
4. The physical network routes the standard UDP packet across physical switches to Node 2.
5. Node 2's kernel decapsulates the outer UDP packet, extracts the original inner IP packet, and routes it directly into Pod B's private Network Namespace.

> **Maximum Transmission Unit (MTU) Caveat:** VXLAN packet encapsulation adds a 50-byte header to every packet (8 bytes UDP + 8 bytes VXLAN + 20 bytes outer IP + 14 bytes outer Ethernet). If the host MTU is 1500 bytes, the CNI interface inside the Pod must be configured with an MTU of 1450 bytes. Failing to account for this MTU reduction leads to silent packet fragmentation, connection drops, and severe TLS handshake failures.

3.2 Direct Cloud Routing CNI: AWS VPC CNI Plugin

In public cloud environments like AWS, overlay encapsulation adds unnecessary CPU overhead and makes Pod IPs unreachable from outside the cluster. The **AWS VPC CNI** plugin implements a direct-routed underlay model:

- The plugin attaches secondary Elastic Network Interfaces (ENIs) directly to the underlying Amazon EC2 worker node instances.
- It allocates secondary private IPv4 addresses from the AWS VPC subnet directly to these ENIs.
- When a Pod is scheduled, the CNI assigns one of these real, native AWS VPC IP addresses directly to the Pod's network interface.
- Pods communicate directly across the AWS VPC network fabric without VXLAN encapsulation, enabling seamless integration with AWS RDS databases, on-premises datacenters via AWS Direct Connect, and native AWS Security Groups per Pod.



4. Cluster Ingress & North-South Traffic Routing

In cloud architecture, network traffic is categorized by directionality:

- **North-South Traffic:** Traffic entering the cluster from external clients, mobile applications, or the internet into internal services.
- **East-West Traffic:** Communication occurring internally between microservices, databases, and background workers within the cluster boundary.

4.1 The Ingress Lifecycle and Path-Based Routing

North-South traffic entering Kubernetes is governed by the **Ingress** resource and executed by an **Ingress Controller** (such as NGINX Ingress or AWS Load Balancer Controller):

1. **DNS Resolution:** An external user navigates to `https://api.example.com/checkout`. Public DNS resolves the domain name to the public IP address of an external Load Balancer (e.g., AWS Application Load Balancer).
2. **TLS Termination:** The Load Balancer terminates the incoming TLS handshake using an SSL certificate managed by AWS Certificate Manager (ACM) or cert-manager, offloading CPU-intensive cryptographic decryption.
3. **Ingress Controller Routing:** The decrypted HTTP request is forwarded to the Ingress Controller Pods running inside the cluster. The controller evaluates its routing tables:
   - Host matching: `api.example.com`
   - Path matching: `/checkout` → routed to Kubernetes Service `checkout-service:8080`
   - Path matching: `/users` → routed to Kubernetes Service `user-service:8080`
4. **Direct Pod Dispatch:** Modern ingress controllers bypass `kube-proxy` ClusterIP translation and route traffic directly to individual backend Pod IPs using EndpointSlice metadata, reducing network hops and latency.

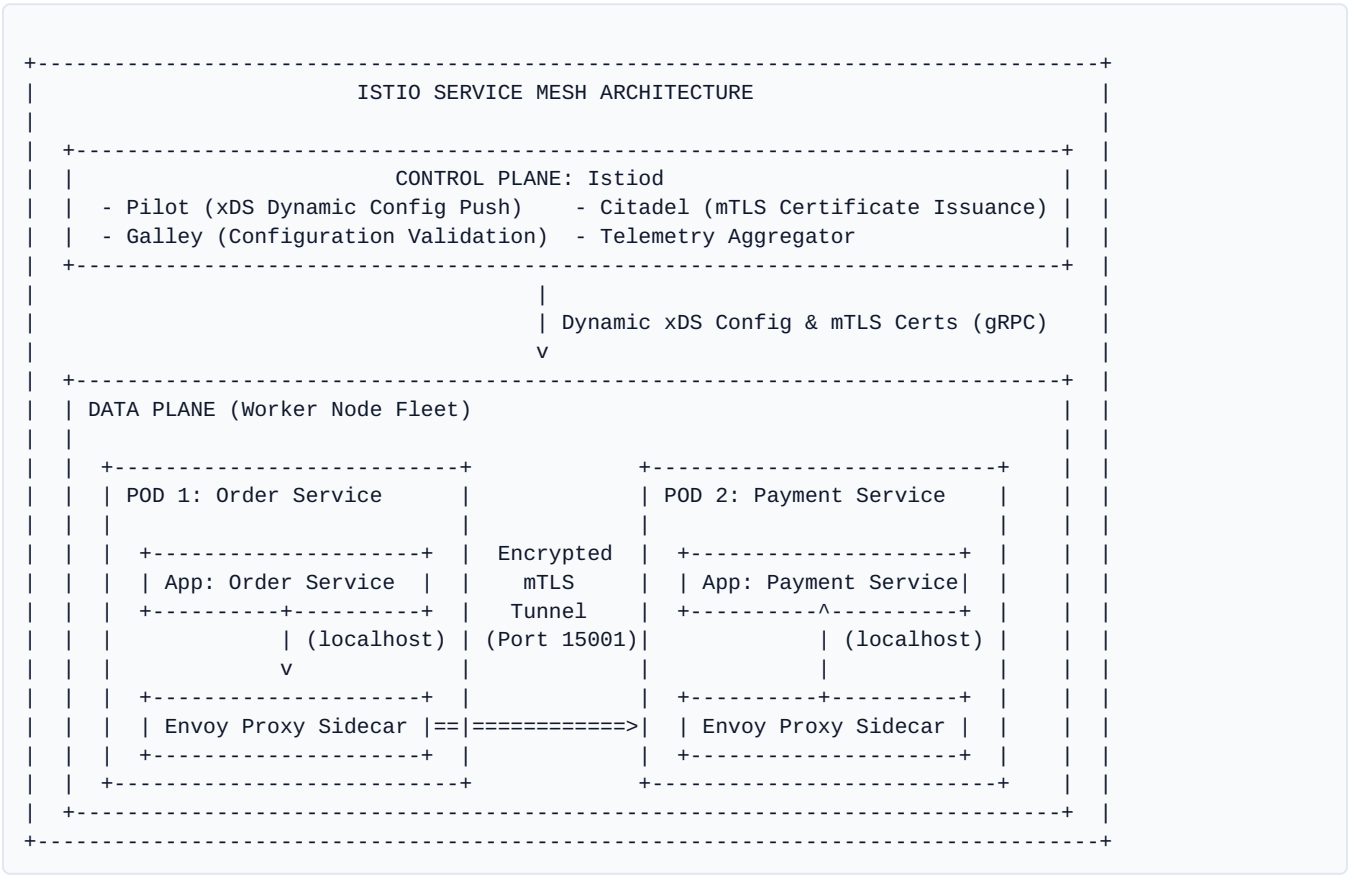
5. Service Mesh Architecture: Istio & Envoy Proxy

While Ingress Controllers secure North-South entry points, a **Service Mesh** secures, manages, and observes all internal East-West microservice traffic.

5.1 The Two-Tier Mesh Architecture

A Service Mesh is strictly divided into two distinct architectural planes:

- **Data Plane:** A fleet of high-performance sidecar proxies (predominantly **Envoy Proxy**) injected directly into every application Pod. The Envoy proxy intercepts all incoming and outgoing TCP, HTTP/1.1, HTTP/2, and gRPC traffic for that Pod. The application code is completely unaware of the proxy.
- **Control Plane (Istiod):** The centralized brain of the mesh. Istiod translates high-level routing rules and security policies into low-level Envoy configuration APIs (Discovery Services / xDS) and dynamically streams them to all Envoy proxies over gRPC. Istiod also operates as a built-in Certificate Authority (CA) to automate the issuance and rotation of cryptographic certificates for mutual TLS.



5.2 Sidecar Injection & iptables Interception Mechanics

How does Envoy intercept traffic without requiring developers to change their application endpoints?

1. **Automatic Sidecar Injection:** A Kubernetes Mutating Admission Webhook detects namespaces labeled with `istio-injection=enabled`. When a Pod is created, the webhook automatically injects an `istio-init` init container and an `istio-proxy` (Envoy) sidecar container into the Pod definition.
2. **iptables Trapping:** The `istio-init` container runs with `CAP_NET_ADMIN` privileges prior to application startup. It configures the Pod's private network namespace iptables rules (specifically the `PREROUTING` and `OUTPUT` chains) to redirect all inbound and outbound TCP traffic to Envoy's local listening port (port 15001 / 15006).
3. **Transparent Mediation:** When the application sends a request to `http://payment-service`, the kernel redirects the socket connection directly to Envoy. Envoy terminates the local connection, verifies routing rules, establishes an encrypted mTLS connection to the destination Pod's Envoy proxy, and forwards the payload.

6. Mutual TLS (mTLS) & Zero-Trust Mesh Security

In standard container environments, traffic between Pods travels across the network in cleartext, creating severe vulnerabilities if an attacker compromises a single container on the node.

A Service Mesh implements a **Zero-Trust Network Architecture** using **Mutual TLS (mTLS)**:

- **Cryptographic Workload Identity (SPIFFE):** Istio assigns every ServiceAccount a unique SPIFFE ID (Secure Production Identity Framework for Everyone) encoded into an X.509 certificate (e.g., `spiffe://cluster.local/ns/prod/sa/order-service-sa`).
- **Automatic Certificate Rotation:** Istiod automatically issues short-lived cryptographic certificates to Envoy sidecars and rotates them continuously every few hours without human intervention.
- **Dual-Sided Authentication:** When Service A connects to Service B, both Envoy proxies exchange certificates. Service A verifies the identity of Service B, and Service B verifies the identity of Service A.

- **Fine-Grained Authorization Policies:** Security teams define declarative `AuthorizationPolicy` resources (e.g., "Allow only workloads with SPIFFE ID `order-service-sa` to issue HTTP POST requests to `/charge` on `payment-service`; deny all other callers").

7. Advanced Traffic Management & eBPF Evolution

Traffic Capability	Architectural Mechanism	Operational Benefit	Real-World Cloud Use Case
Canary Traffic Shifting	Istio `VirtualService` splits traffic percentages at Layer 7 (e.g., 90% v1, 10% v2).	Zero-downtime testing of new releases with minimal blast radius.	Gradual rollout of a new microservice version in Amazon EKS.
Circuit Breaking	Envoy tracks consecutive 5xx errors and temporarily evicts unhealthy upstream hosts.	Prevents cascading microservice outages and thundering herd failures.	Halting traffic to an overloaded downstream billing database.
Distributed Tracing	Envoy injects and propagates B3/W3C trace headers (`x-request-id`, `x-b3-traceid`).	End-to-end request latency visibility across deep microservice call graphs.	Tracing slow database queries using OpenTelemetry and AWS X-Ray.
eBPF Data Plane (Cilium)	Extended Berkeley Packet Filter (eBPF) programs injected directly into the host Linux kernel.	Bypasses host TCP/IP stack and iptables overhead; enables sidecarless mesh.	High-throughput, ultra-low latency container networking on AWS Graviton instances.

8. Practical Operational Troubleshooting

Troubleshooting complex container networking and service mesh failures requires inspecting packet flows, CNI logs, and Envoy proxy configuration states.

Failure Symptom	Root Cause	Diagnostic Commands	Remediation Action
Connection Timeout on Large Payloads	MTU mismatch between overlay network (VXLAN 1450 bytes) and physical host (1500 bytes), causing packet drops.	`ping -M do -s 1472 <remote-ip>` `ip link show`	Reduce MTU setting in CNI configuration (e.g., set Calico MTU to 1440 or 1450).
503 Service Unavailable (Envoy)	Envoy cannot reach upstream service; connection reset or upstream circuit breaker tripped.	`istioctl dashboard envoy <pod>` `kubectl logs <pod> -c istio-proxy`	Verify destination Pod health, verify Service port name conventions (e.g., `http-web`), and check mesh DestinationRules.
Pod IP Exhaustion	AWS VPC CNI exhausted available private IP addresses within the assigned EC2 VPC subnet.	`kubectl describe node <node>` (Inspect CNI events)	Add secondary CIDR blocks to AWS VPC or enable AWS VPC CNI Custom Networking.
mTLS Handshake Failure	Certificate expiration, SPIFFE trust domain mismatch, or conflicting PeerAuthentication modes.	`istioctl authn tls-check <pod>` `istioctl proxy-status`	Re-synchronize Istiod certificates; ensure client and server use matching `STRICT` or `PERMISSIVE` mTLS modes.

9. Explicit Architectural Relationships & Dependencies

This section explicitly documents the structural dependencies and multi-hop interactions across cloud networking and service mesh components. These relational facts are optimized for Graph RAG entity linking and traversal:

- **`Kubernetes Kubelet executes CNI plugins:`** The Kubelet invokes local CNI binaries during Pod initialization to create veth pairs and configure network namespaces.
- **`AWS VPC CNI integrates with AWS VPC:`** The AWS VPC CNI plugin allocates native AWS VPC secondary IP addresses directly to Amazon EKS Pod interfaces.
- **`VXLAN implements Overlay Networks:`** VXLAN encapsulates Layer 2 container Ethernet frames inside Layer 4 UDP underlay packets (port 4789).
- **`Ingress Controllers route North-South traffic:`** An Ingress Controller terminates external TLS connections and directs incoming client requests to internal Kubernetes Services based on URL paths.
- **`AWS Load Balancer Controller provisions AWS ALB:`** The controller watches Kubernetes Ingress objects and automatically provisions managed AWS Application Load Balancers.
- **`Istio Service Mesh controls Envoy proxies:`** The Istiod control plane uses the xDS gRPC API to dynamically distribute routing configurations and mTLS certificates to Envoy sidecars.
- **`Envoy Proxy intercepts application traffic:`** iptables rules in the Pod network namespace transparently redirect inbound and outbound TCP traffic into the Envoy sidecar.
- **`Mutual TLS (mTLS) enforces Zero-Trust security:`** Envoy sidecars exchange SPIFFE-validated X.509 certificates to authenticate service identities and encrypt all East-West traffic.
- **`eBPF accelerates Container Networking:`** Modern CNI architectures like Cilium run eBPF bytecode programs inside the host Linux kernel to bypass iptables and route container packets at line speed.
- **`Amazon EKS supports Istio and AWS App Mesh:`** Amazon EKS clusters run Istio and Envoy sidecar proxies on Amazon EC2 instances to secure microservice architectures.

10. Key Takeaways, Entities & Interview Questions

10.1 Key Takeaways

- Container Network Interface (CNI) plugins provide the underlying network plumbing for Kubernetes, implementing either overlay encapsulation (VXLAN) or direct cloud routing (AWS VPC CNI).
- North-South traffic entering the cluster is managed by Ingress Controllers, providing TLS termination, virtual hosting, and Layer 7 URL path-based routing.
- East-West microservice traffic is secured and managed by a Service Mesh (Istio), which deploys Envoy sidecar proxies to intercept application network traffic transparently.
- Service Meshes implement Zero-Trust security by providing cryptographic workload identities (SPIFFE) and automated mutual TLS (mTLS) encryption.
- Modern cloud networking is shifting toward eBPF (Cilium), replacing legacy iptables packet filtering with high-performance kernel bytecode programs.

10.2 Important Entities & Classification

- **Networking Standards & Interfaces:** CNI (Container Network Interface), IPAM, SPIFFE, xDS API, MTU, VXLAN, Geneve.
- **CNI Plugins:** AWS VPC CNI, Calico, Cilium, Flannel.
- **Ingress & North-South Controllers:** Ingress, Ingress Controller, NGINX Ingress, AWS Load Balancer Controller, AWS ALB/NLB.
- **Service Mesh Components:** Service Mesh, Istio, Istiod, Envoy Proxy, Data Plane, Control Plane, Sidecar Pattern.
- **Security & Encryption Primitives:** Mutual TLS (mTLS), X.509 Certificates, SPIFFE ID, PeerAuthentication, AuthorizationPolicy.

- **Kernel Networking Primitives:** Linux Network Namespaces, veth pairs, iptables, nftables, eBPF (Extended Berkeley Packet Filter).
- **Cloud Infrastructure:** AWS VPC, Amazon EC2, Amazon EKS, Elastic Network Interface (ENI), AWS Certificate Manager (ACM).

10.3 Common Technical & Interview Questions

1. **What is the fundamental architectural difference between an Overlay CNI (e.g., VXLAN) and a Direct Cloud CNI (e.g., AWS VPC CNI)?**
   Answer: An Overlay CNI encapsulates container packets inside standard UDP packets (using VXLAN), creating a private virtual network on top of the physical network at the cost of MTU overhead and CPU processing. A Direct Cloud CNI allocates real, routable VPC IP addresses directly to Pods via EC2 Elastic Network Interfaces (ENIs), eliminating encapsulation overhead and allowing Pods to communicate natively with cloud resources.
2. **How does a Service Mesh sidecar intercept application traffic without requiring code modifications?**
   Answer: During Pod creation, an init container (`istio-init`) executes with `CAP_NET_ADMIN` privileges to configure the Pod's private Linux Network Namespace `iptables` rules. Inbound and outbound TCP traffic hitting the network namespace is redirected to the Envoy sidecar proxy listening on local ports (15001/15006).
3. **What is Mutual TLS (mTLS), and how does Istio automate it across microservices?**
   Answer: Mutual TLS is a protocol where both the client and server validate each other's cryptographic identities before establishing an encrypted tunnel. Istiod acts as a Certificate Authority, issuing short-lived X.509 certificates with SPIFFE identities to Envoy sidecars. The Envoy proxies handle the bidirectional TLS handshake and certificate validation transparently.
4. **What causes MTU-related packet drops in overlay networks, and how is it resolved?**
   Answer: VXLAN encapsulation adds 50 bytes of header data to every packet. If the host network has a standard 1500-byte MTU, encapsulating a full 1500-byte container packet results in a 1550-byte frame that exceeds physical network limits, causing dropped packets. The issue is resolved by reducing the container CNI interface MTU to 1450 bytes.
5. **How does eBPF improve upon traditional iptables-based container networking?**
   Answer: iptables requires sequential evaluation of packet filtering rules, introducing latency in large clusters with thousands of Pods and Services. eBPF executes sandboxed bytecode programs directly within the Linux kernel at network socket and driver layers, providing $O(1)$ hash table lookups, direct packet forwarding, and bypassing kernel TCP/IP stack overhead.